

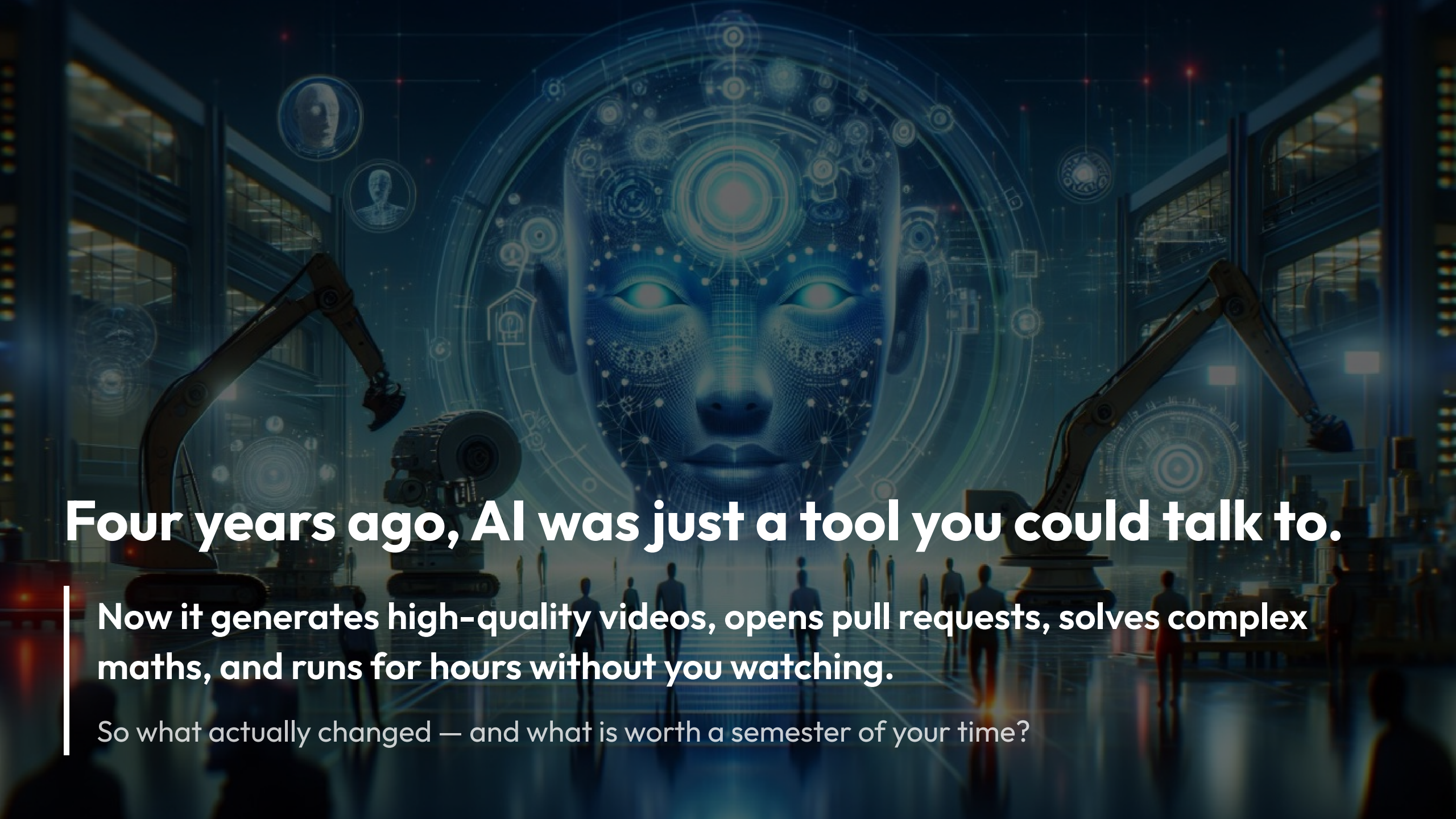
Course Introduction

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University



Four years ago, AI was just a tool you could talk to.

Now it generates high-quality videos, opens pull requests, solves complex maths, and runs for hours without you watching.

So what actually changed — and what is worth a semester of your time?

Motivation — why study LLM agents?

There are at least three major reasons:

BETTER WORK WITH LLMS

Not knowing how an LLM works costs you most of it. Same model, different ceiling.

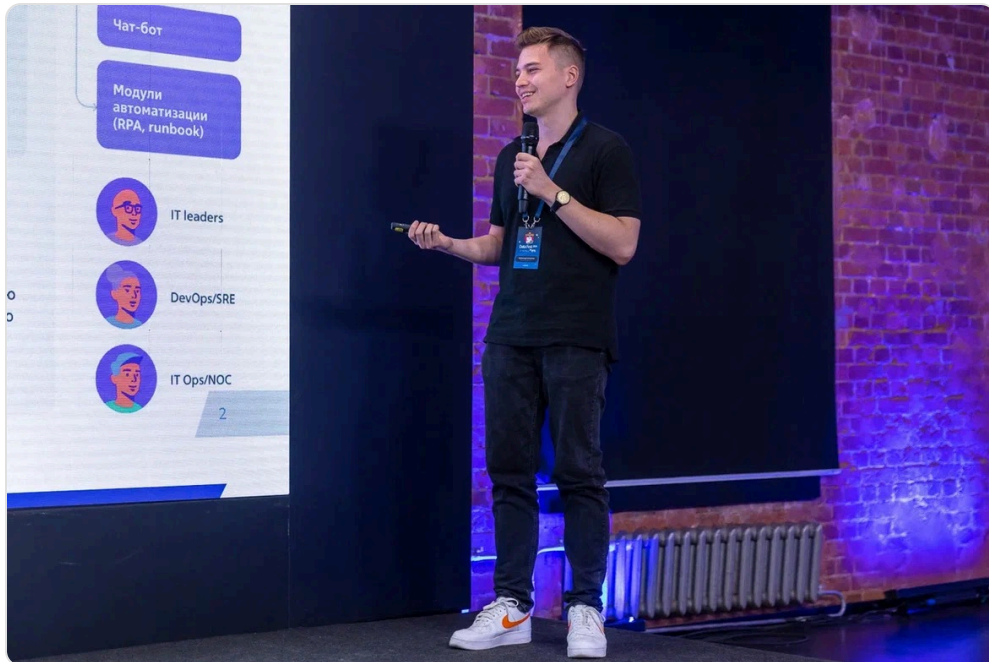
MAXIMUM PRODUCTIVITY, THROUGH THE HARNESS

Tools, skills, memory, context — the layer around the model, and its name is the **harness**. Knowing what each part does is knowing which one to reach for.

BE THE ONE WHO DECIDES HOW IT IS USED

AI is spreading into every field. Whoever understands it is who sets how it gets used there.

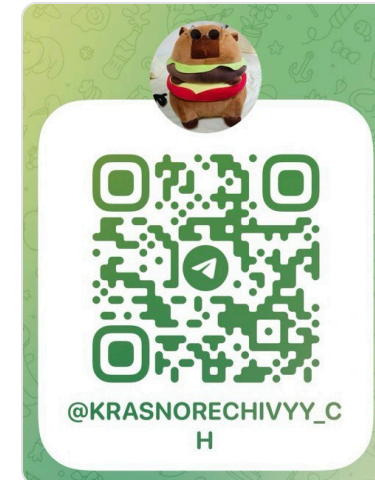
About Me



- **4.5+ years** of applied AI experience
- **Now** — LLM applications and AI agents at **Sberbank**
- **Before** — AI Researcher and MSc student at **Tsinghua**, ML Engineer at **VTB**
- Graduated **HSE** in 2024

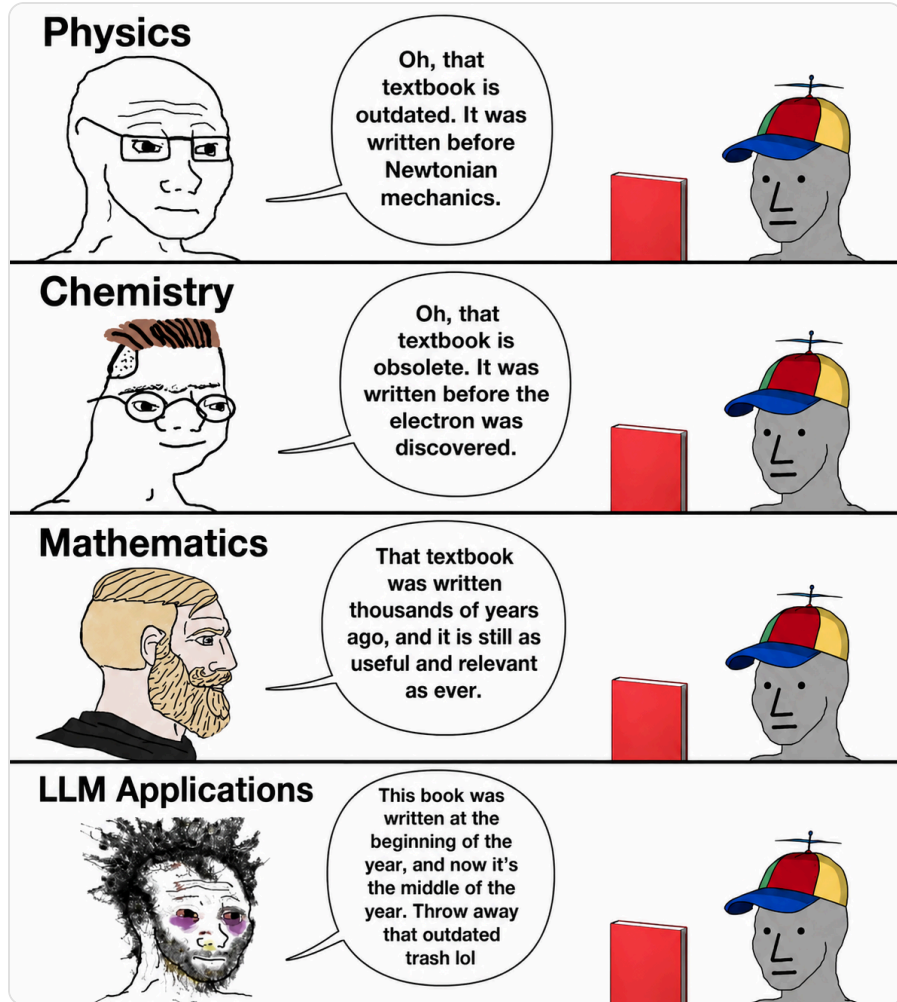


Me



My channel

The frameworks expire, the ideas stay



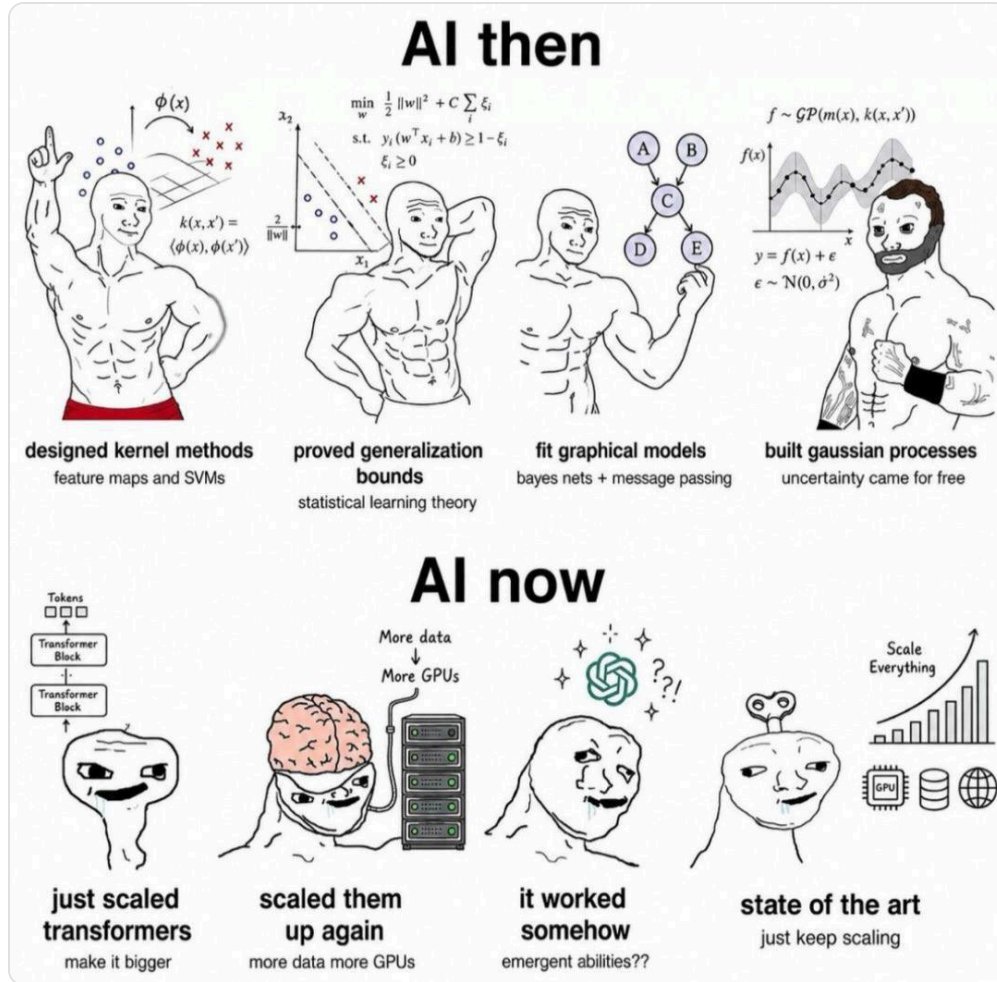
AI IS DEVELOPING VERY FAST

The tooling has a half-life of about a year. What was standard practice last spring is legacy now, and the framework at the centre of every tutorial changes faster than a syllabus can.

WHY STUDY CONCEPTS OVER TOOLS

A framework you learn expires with it. The ideas underneath — a loop, a tool contract, a decision about what stays in context — are what the *next* framework will also be built from.

Trend for Versatility



HOW WE USED TO BUILD

Break the problem into small pieces and solve each one separately: a classifier here, a rules engine there, a hand-written pipeline to join them. Every new problem meant a new system.

WHAT WE DO NOW

One large general model, given the right tools and skills, pointed at whatever the problem is. The engineering moved from **designing the solution** to **equipping the model** — and that shift decides more about who ships than most people expect.

DISCUSSION

Two people are building the same video game. Both know exactly what they want to build.

One knows game development well — engines, physics, level design — and for AI uses ChatGPT in a browser tab only.

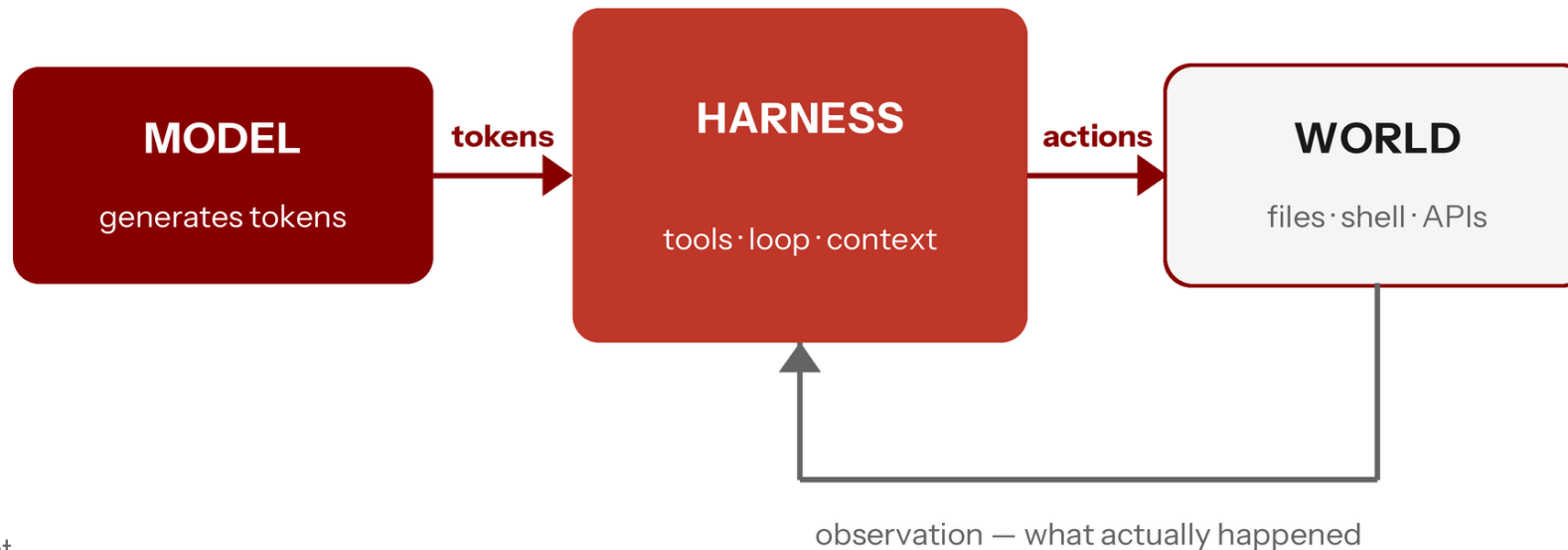
The other barely knows the domain, but works inside an agent harness: git and a real repository structure, deliberate context management, the right tools and skills at each step.

Who is more likely to succeed?

Agent Harness

An **agent harness** is the software around a language model that gives it tools, decides what goes into its context, executes what it asks for, and feeds the result back. The model itself is unchanged — everything here is engineering around it.

Model · Harness · World



How we got here

Six waves in four years. The chronology is not the point — the point is that **no wave replaced the one before it**. A 2026 universal agent still runs the ReAct loop from 2023, still retrieves, still calls functions.

Six Waves — and each one kept the last

01 · JUST AN LLM · 2022-23

one prompt, one answer. No memory, no tools

02 · ReAct · 2023

reason, call a tool, look at the result, repeat

03 · CHAINS, RAG, SO · 2023-24

retrieval, function calling, pipelines

04 · MULTI-AGENT · 2024-25

several models with roles, and a supervisor

05 · UNIVERSAL AGENTS · now

files, environment, MCP, skills. One agent, any task

06 · WHAT COMES NEXT

self-improving loops, long autonomous runs

Inside a good harness

THE LOOP ITSELF

- **System policy** — role, rules, response format, stop conditions
- **Tool contracts** — schemas, allow-lists, argument and result validation
- **Runtime loop** — parse the tool call, execute, observe, decide, repeat
- **Context management** — trimming, summaries, files, subagents

EVERYTHING AROUND IT

- **Common interfaces** — `AGENTS.md`, skills, MCP as standard channels
- **Reliability and safety** — retries, timeouts, budgets, sandboxing, approvals
- **Product shell** — CLI, API, auth, run history, logs, observability

Every part of it is something you are about to learn

Harness component	Underneath it is	Lecture
Runtime loop	ReAct — thought, action, observation	04
Tool contracts	function calling, JSON schema, validation	05
Context management	retrieval, summarisation, memory tiers	06 · 07
Subagents	orchestration, context isolation	08
System policy	prompting, chain-of-thought, spec files	10 · 11
Reliability	retries, timeouts, budgets, prompt injection	05 · 13
Product shell	serving, streaming, observability, cost	03 · 13

Nothing in the middle column is new. **The harness is the assembly, not the invention** — which is exactly why the parts are worth knowing.

So why is this course called Multi-Agent Systems?

WHAT DIED

Seven agents with personalities debating in a group chat until they agreed. It produced excellent transcripts and very few results.

WHAT WON

One entry point. Inside it, an orchestrator spawning subagents that each get a clean context and hand back only a conclusion.

AND IT IS GENUINELY UNSETTLED

Anthropic ships orchestrator-plus-subagents and reports large gains. Cognition published *Don't Build Multi-Agents* and argues for a single thread. Both are serious, both are public, and **Lecture 08 is where we work out who is right.**

The course

- I Foundations** — 01 project setup and tooling · 02 how an LLM works · 03 how it is served
- II The agent and its parts** — 04 what an agent is · 05 tools and structured output · 06 RAG · 07 memory
- III Systems** — 08 multi-agent systems · 09 frameworks · 10 advanced reasoning
- IV Practice and frontier** — 11 agentic coding · 12 real systems and their evidence · 13 AgentOps · 14 frontiers

Fourteen lectures, eighty minutes each. Every one of them is a row from two slides ago.

The project

$$\text{grade} = 0.4 \times \text{proposal} + 0.6 \times \text{defense}$$

Teams of four. Each stage is marked out of 10.

PROPOSAL · 10

- **4** — a valid idea. Why does this need an LLM at all?
- **4** — a working MVP: 1 it runs live, 1 a real repository, 2 the core LLM feature
- **2** — answers to technical questions

DEFENSE · 10

- **4** — finished: 2 deployed and reachable by link, 2 matches what you proposed
- **4** — tooling and technical depth
- **2** — answers to technical questions

What "technical depth" actually means

1 POINT

A single LLM call and nothing around it. A repository with no README and no Dockerfile.

4 POINTS

Several agents with real communication between them, asynchronous request handling, function calling — used because the problem needed them.

THE CAVEAT THAT MATTERS MOST

This is not know-how for its own sake — we are not writing a research paper. A simple project with clean, well-structured code loses nothing for having no function calling, **as long as you can say why you did not need it**. Justify the choice in the presentation or in questions, and it counts.

Summary

WHAT WE COVERED

- What an agent harness is, and the six waves that produced it
- That every component inside it is older than it is
- Where each of those components is taught in this course
- The project, the rubric, and the dates

WHAT TO RETAIN

- The harness is an **assembly of known parts**, not a new invention
- Hold the model fixed and the rig still moves the result — sometimes past the gap between two models
- "One agent" describes an *interface*. Open it and it is still multi-agent

References

- Yao et al. 2023 · **ReAct: Synergizing Reasoning and Acting in Language Models** · arXiv 2210.03629
- Yang et al. 2024 · **SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering** · arXiv 2405.15793
- **The Binding Constraint Thesis** — the harness as a determinant of agent performance · arXiv 2605.23950
- **Harness decomposition and model-harness coupling** · arXiv 2606.20683
- Anthropic · **How we built our multi-agent research system** · anthropic.com/engineering
- Cognition · **Don't Build Multi-Agents** · cognition.com/blog

What's Next — Lecture 01

PRODUCTION ML PROJECT SETUP

The unglamorous half of every agent project: the shell it runs in, the history it keeps, and the container it eventually ships in.

It comes first for a reason. Two of the four marks on your defense are for a repository someone else can clone and a service someone else can reach — neither of which is about agents at all.

IN IT

- Shell — navigation, pipes, `grep` and `find`, permissions, environment variables
- Git — commits, branches, and resolving a conflict without panic
- The files a repository needs, and keeping secrets out of all of them
- Docker, and why "it works on my machine" is not a result

Thank You

Questions & Discussion

Lecture 00 · LLM Agent Development